Module 2: Security & Guardrail Layer

A deterministic, fully testable safety layer that sits between the LLM/agents and any device executor.

Design Philosophy

Every decision is code, never a model's judgment. An LLM told "don't touch banking apps" in a prompt can be tricked by adversarial input. A hard-coded check in the execution path cannot.

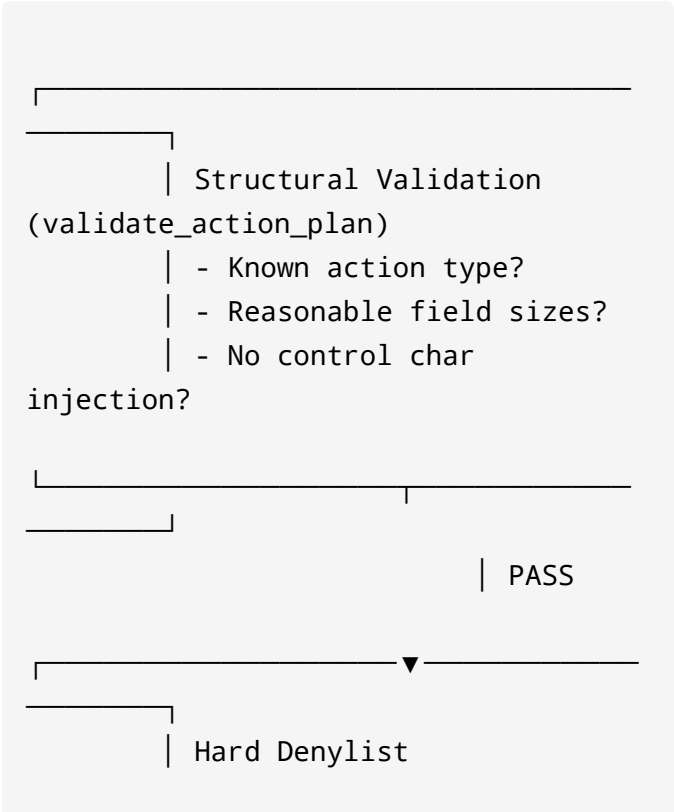
No exceptions for urgency or justification. A voice command saying "please, I trust you" does not override the guardrail — only explicit human confirmation (single-use, replay-safe) can.

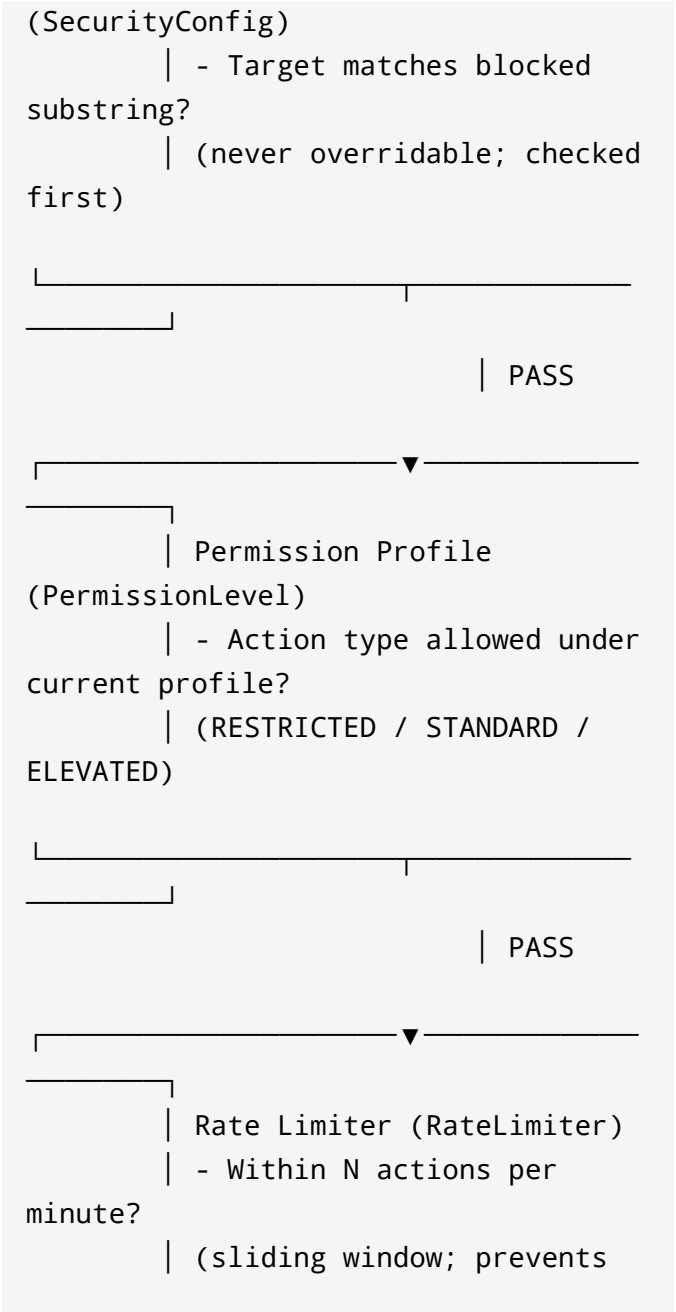
Fail closed, never open. An unknown action type, a malformed plan, a rate limit exceeded —

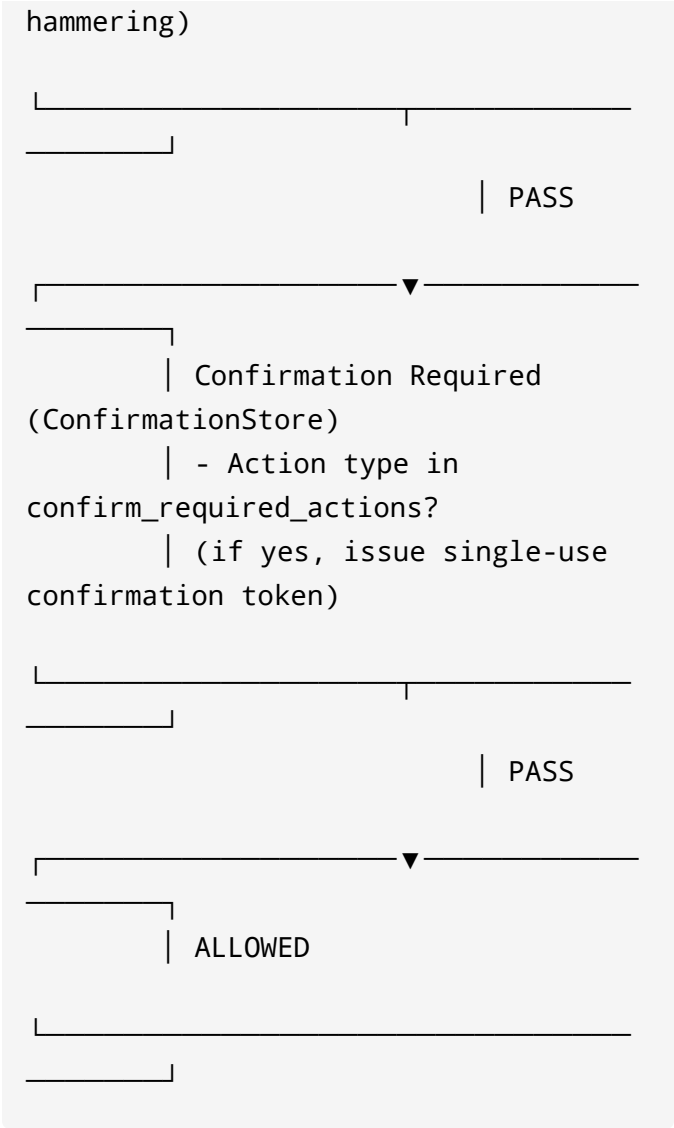
these all result in BLOCKED, never silently allowed because "we're not sure."

Architecture: Five-Stage Evaluation Pipeline

Every `ActionPlan` passes through these stages in order. The first match wins; later stages are never reached:




(SecurityConfig)
- Target matches blocked substring?
(never overridable; checked first)
PASS
Permission Profile (PermissionLevel)
- Action type allowed under current profile?
(RESTRICTED / STANDARD / ELEVATED)
PASS
Rate Limiter (RateLimiter)
- Within N actions per minute?
(sliding window; prevents


hammering)
PASS
Confirmation Required
(ConfirmationStore)
- Action type in
confirm_required_actions?
(if yes, issue single-use
confirmation token)
PASS
ALLOWED

Modules

`validation.py`

- `validate_action_plan(plan)` — Structural checks only. Raises `PlanValidationError` on malformed/unknown action type/oversized field.
- `KNOWN_ACTION_TYPES` — Exhaustive registry. An action type reaching the guardrail that isn't here is rejected full stop.

Why this exists: An LLM planner can hallucinate action types that don't have an executor or policy. Failing on unknown types prevents both accidental and adversarial attempts to execute non-existent actions.

`permissions.py`

- `PermissionProfile` — A named, coarse permission level.
  - `RESTRICTED_PROFILE` — blocks dangerous actions (delete, power, call, code deployment).
  - `STANDARD_PROFILE` — blocks only `deploy_code` (local testing).

    - `ELEVATED_PROFILE` — blocks nothing (but confirmation tier still applies).
- `PermissionLevel` enum — RESTRICTED, STANDARD, ELEVATED.

Use case: Switch profiles at runtime (e.g., entering "guest mode" when lending the device to someone).

`rate_limiter.py`

- `RateLimiter` — Sliding-window rate limiter.
- Configurable: `max_actions_per_minute`, adjustable clock (for testing).
- Thread-safe.

Why sliding window (not fixed): A fixed-window limiter lets callers burst 2x the limit across a boundary. Sliding window bounds to N actions over ANY 60-second window.

`confirmation.py`

- `ConfirmationStore` — Manages single-use, expiring (TTL default 120s) confirmation tokens.

- **Key insight: Replay safety.** A confirmation token is bound to the exact content hash of ONE plan. If the plan is mutated (accidentally or via prompt injection) between description and execution, redemption fails.

Example flow:

1. `evaluate()` returns `REQUIRES_CONFIRMATION` with an issued `confirmation_id`.
2. User is shown a description of the plan (e.g., "Send message 'hi' to Sarah").
3. User says "confirm" or clicks yes.
4. Caller presents the **exact same** plan + the `confirmation_id` to `guardrail.confirm()`.
5. If plan content matches (via hash), confirmation is redeemed and the action is ALLOWED.

`audit.py`

- `AuditLogger` — Appends one JSON line per guardrail decision to a dedicated file.

- Separate from general application logs (different retention, different consumers).
- Every decision — ALLOWED, BLOCKED, REQUIRES_CONFIRMATION — is recorded.

Format: One JSON object per line.

```
{"timestamp": "2026-07-19T...",
"decision": "allowed", "action_type":
"tap", "target": "com.example", ...}
```

`encryption.py`

- `FernetEncryptionProvider` — Production encryption (cryptography.Fernet, AES-128-CBC + HMAC-SHA256).
- `NullEncryptionProvider` — No-op passthrough for local dev.
- `build_encryption_provider()` — Factory enforcing that production never silently falls back to unencrypted.

Wired into config: `NEXUS_ENCRYPTION_KEY` (env var), checked at startup in production.

`guardrail.py`

- `DeterministicGuardrail` — Implements `core.interfaces.GuardrailEngine`.
- Orchestrates all five stages above.
- `evaluate(plan)` — Returns `GuardrailResult` (ALLOWED | BLOCKED | REQUIRES_CONFIRMATION).
- `confirm(confirmation_id, plan)` — Redeems a confirmation token.
- `evaluate_or_raise(plan)` — Exception-based convenience wrapper for when exceptions are preferred over switch statements.
- `set_permission_profile(profile)` — Runtime profile switching.

Testing: 71 Tests, Comprehensive Coverage

Module 2 introduces 71 new tests covering:

- **validation:** malformed plans, unknown action types, oversized fields, control char injection

- **permissions:** profile enforcement, runtime switching
- **rate_limiter:** sliding window, expired actions, concurrency
- **confirmation:** single-use tokens, replay safety, plan hashing, TTL expiry
- **audit:** JSON serialization, concurrent writes, record retention
- **encryption:** roundtrip encryption/decryption, key validation, production enforcement
- **guardrail:** all five evaluation stages, precedence order, integration tests

All 105 cumulative tests (Module 1 + 2) pass.

Usage Example

```
from nexus.core.config import
load_config
from nexus.security import
DeterministicGuardrail
from nexus.core.interfaces import
ActionPlan, GuardrailDecision

config = load_config()
```

```
guardrail =
DeterministicGuardrail(security_confi
g=config.security)

# An agent proposes an action
plan = ActionPlan(
    action_type="send_message",
    target="com.whatsapp",
    params={"text": "hello sarah",
"contact": "sarah@example.com"},
)

# Evaluate it
result = guardrail.evaluate(plan)

if result.decision ==
GuardrailDecision.ALLOWED:
    # Execute the action
    executor.execute(plan)
elif result.decision ==
GuardrailDecision.REQUIRES_CONFIRMATI
ON:
    # Issue a confirmation prompt to
the user
    # User responds:
guardrail.confirm(result.confirmation
_id, plan)
    print(f"Awaiting human
confirmation: {result.reason}")
```

```
    print(f"Confirmation ID:
{result.confirmation_id}")
elif result.decision ==
GuardrailDecision.BLOCKED:
    # Explain the rejection to the
user
    logger.warning(f"Action blocked:
{result.reason}")
```

Or using the exception-based API:

```
try:
    plan =
guardrail.evaluate_or_raise(plan)
    executor.execute(plan)
except GuardrailViolation as exc:
    logger.error(f"Action blocked:
{exc}")
except ConfirmationRequiredError as
exc:
    confirmation_id =
exc.details["confirmation_id"]
    # prompt user, then:
guardrail.confirm(confirmation_id,
...)
```

Dependencies

Runtime (core): none (standard library only).
Runtime (security): `cryptography` (optional; installed as `nexus-assistant[security]`).
Dev/test: `pytest`, `mypy`, `ruff` (all optional).

Key Design Decisions

1. **Deterministic, testable, no randomness.** Every decision reproduces identically given the same input. Tests can mock time/clocks to verify TTL, rate limiting, etc.
2. **Single permission profile at a time.** NEXUS is a single-user personal assistant, not a multi-tenant system. One call to `set_permission_profile()` changes the active mode for all subsequent evaluations.
3. **Confirmation tokens are short-lived (2 min default).** A confirmation prompt that hangs around for hours is a security liability (token capture, forgotten confirmations, accidental redemptions).
4. **Audit trail is append-only, never pruned.** Operators are responsible for log rotation/archival. NEXUS's audit log should

ship to real log-aggregation infrastructure in production.

5. **No "emergency override" mode.** You don't get to whisper "please just execute this, I trust you" to bypass the guardrail. The guardrail is the entire point.

Next Module

Module 3 candidate: **LLM Brain** (`nexus.agents` + provider adapters for Claude/GPT/Gemini/Ollama), since everything downstream depends on having a working planner LLM.

Alternatively: **Memory Layer** (`nexus.memory`) for long-term retrieval, if you'd prefer to have storage available before wiring up the agents.

Confirm direction and I'll proceed.